

Continuous Colony Health Monitoring for Apiaries Using a Fog-to-Cloud Architecture

Bhaskar

MSc in Cloud Computing

Fog and Edge Computing (H9FECC)

National College of Ireland

Student ID: X24262404

x24262404@student.ncirl.ie

Abstract—A honeybee colony can slip from healthy into starvation, chilled brood, or the run-up to a swarm within days, yet an apiary is often inspected only every week or two and is frequently sited far from the beekeeper. This paper presents a working fog-to-cloud pipeline that watches two apiaries continuously and, rather than only report raw numbers, reasons about colony state. Each hive carries five instruments, weight, brood-nest temperature, internal humidity, acoustic buzz frequency, and entrance traffic, that are sampled locally and dispatched to a fog gateway at the apiary. The gateway folds each stream into a fixed time window, reduces it to summary statistics, and evaluates colony rules that flag brood overheating, brood chilling, starvation, and the acoustic signature that precedes swarming. Compact per-window aggregates cross a managed message queue into an event-driven ingestion function that stores them; a second, independently deployed function serves a dashboard programming interface that turns the stored windows into a plain-language colony-health summary for each apiary, and the operator page is delivered as static content from object storage. The system is built in Node.js and provisioned to a real Amazon Web Services account by a single infrastructure-as-code apply that created twenty-four resources. Measurement against the live endpoint shows the pipeline healthy end to end within about a minute of start-up, the stored record count rising across polls, and both apiaries rendering live readings, per-hive narratives, and firing brood-overheat alerts in a browser.

Index Terms—fog computing, edge computing, precision apiculture, colony health monitoring, serverless, message queue, Internet of Things.

I. INTRODUCTION

Beekeepers have long faced high and variable colony losses, and a colony's decline is often well advanced before a routine inspection reveals it. Opening a hive is itself disruptive, it chills the brood nest and provokes the bees, so inspection is deliberately infrequent, which leaves long blind intervals during which a queen can fail, stores can run out, or a colony can prepare to swarm. Precision apiculture responds to this by instrumenting the hive so that its state can be observed without opening it, and a body of work has shown that continuously logged signals such as weight, internal temperature, and sound carry rich information about colony condition [1], [2]. An apiary, moreover, is often remote, at the edge of a field or on a rooftop, so the sensing problem is naturally a wireless-sensor-network problem, with the attendant constraints on bandwidth and connectivity [3].

Those constraints make a naive design, streaming every raw reading to a central cloud, a poor fit. A hive scale, a temperature probe, and a microphone together produce readings far faster than any single reading is useful, the uplink from a remote apiary is limited and intermittent, and a monitoring view that fails whenever connectivity drops is of least use in the exposed locations apiaries occupy. Fog and edge computing offer a better division of labour by moving computation toward the data source [4]. Fog computing inserts a compute tier between the hives and the cloud, so that windowing, statistical reduction, and time-critical rule evaluation happen at the apiary, while the cloud retains durable storage, elastic query serving, and a globally reachable interface [5], [6]. The cloud services used here follow the standard on-demand, elastic, measured model of cloud computing [7].

A further observation shapes this work. Even continuous numbers are hard for a beekeeper to act on if they arrive only as five simultaneous graphs per hive; what a keeper wants to know is whether a colony is thriving, stressed, or about to swarm. This system therefore does two things beyond plain telemetry. At the edge it evaluates colony rules, including an acoustic rule, because a colony preparing to swarm changes its sound before it changes its weight. At the read tier it condenses the recent windows for each apiary into a short plain-language health summary, so the dashboard reports a judgement, not just a chart.

This paper reports the design, implementation, and live deployment of such a pipeline. The deployment watches two apiaries, each carrying five instruments. The objectives are fourfold. First, to reduce raw readings to compact per-window aggregates at a fog tier at the apiary. Second, to evaluate colony-health rules at the edge, including the acoustic precursor of swarming, so that a developing problem is flagged in the cycle it appears. Third, to build a scalable cloud back end that ingests aggregates reliably, stores them durably, and serves both the readings and a synthesised colony-health narrative to an operator dashboard without the write path and the read path contending. Fourth, to deploy the whole system to a real cloud account and verify it end to end. The remainder of the paper describes the architecture and its design decisions, the implementation and its tests, the deployment, an evaluation from the running system, and a concluding reflection.

II. SYSTEM ARCHITECTURE AND DESIGN

The system is a linear ingestion pipeline with a separate read path for the operator view. Instruments produce readings; a fog gateway at the apiary windows, summarises, and screens them; a message queue carries aggregates to the cloud; an event-driven function persists them; and an independent function reads the stored data back out, synthesising a colony-health narrative in the process. Fig. 1 shows the deployed topology. This section takes each tier in turn and then weighs the alternatives.

A. Instrumented Hives and the Fog Gateway

Each apiary is represented by five independent sensor processes, one per measured quantity: hive weight in kilograms, brood-nest temperature in degrees Celsius, internal humidity as a percentage, acoustic buzz frequency in hertz, and entrance traffic as a bee count. A sensor generates values with a bounded random walk: each new reading steps a small random amount from the previous one and is clamped to a physically plausible range, so a stream drifts realistically rather than jumping arbitrarily. Every sensor is governed by two independently configurable intervals, one setting how often a value is produced and the other how often the accumulated values are dispatched to the gateway. Separating sampling from transmission lets a fast-changing quantity such as sound be sampled often while still being sent to the edge in economical batches. Each dispatch is a small JSON object naming the sensor type, the apiary identifier, and the unit, plus the short list of timestamped values accumulated since the previous dispatch, a payload of a few hundred bytes.

The fog gateway is a small Node.js and Express web service. It accepts posted batches and keeps one running accumulator per combination of sensor type and apiary. Rather than retain raw readings, it folds each incoming value immediately into a compact record holding the count, running sum, minimum, maximum, and most recent value, so its memory footprint stays flat regardless of how many readings arrive. On a fixed cadence it closes the current window, sealing each accumulator into a summary carrying the window bounds, the count, the minimum, the maximum, the mean, and the latest value, and evaluating that summary against the colony rules before forwarding it.

B. Edge Alert Rules for Colony Health

Four rules are evaluated; internal humidity and entrance traffic are retained for display and narrative but carry no rule, because neither has a single threshold that by itself denotes a fault. Table I lists the rules. Each rule is stored as a tuple whose field names both a sensor type and the aggregate to test, for example the mean brood-nest temperature, so a single evaluator can apply every rule uniformly. Three of the four watch the window mean, because a sustained average is a more reliable signal of a genuine colony condition than one noisy sample. The swarming rule watches the mean acoustic buzz frequency: a colony preparing to swarm raises the pitch of its collective sound, so an elevated mean frequency

TABLE I
EDGE-EVALUATED COLONY-HEALTH RULES

Signal	Rule (per window)	Alert raised
Brood temperature	mean above 36 °C	Brood overheating
Brood temperature	mean below 32 °C	Brood chilling
Hive weight	mean below 20 kg	Colony starvation
Acoustic buzz	mean above 350 Hz	Swarming precursor
Humidity, traffic	(informational, no rule)	None

is an early, non-invasive indicator that precedes the visible signs. Evaluating these rules at the edge means a developing condition is flagged in the same cycle as the readings that reveal it, without a round trip to the cloud.

When a window closes the gateway does not send one message per hive per stream. It collects every summary produced in that cycle and hands the whole set to a single batched send toward the queue, splitting the set into chunks only where the queue interface caps the number of entries in one batched call. For a two-apiary deployment producing up to ten summaries per cycle, this turns ten network calls to the cloud into one, lowering request cost and per-message overhead.

C. Queue-Decoupled Ingestion and the Reading Store

Aggregates leave the fog tier through a managed message queue rather than a direct call into the storage function. The queue is a buffer and a shock absorber: it holds messages durably until consumed, so a slow or briefly unavailable consumer neither loses data nor pushes back on the gateway, and a burst of aggregates is smoothed to a rate the consumer can sustain, the decoupling that message-oriented middleware provides between producers and consumers [8]. The decoupling covers the consumer side of the queue, not the hop onto it: if the gateway's batched send fails, the window has already been sealed and cleared, so the gateway logs the failure and moves on rather than retrying or buffering locally, a stated trade-off accepted at the data volume a two-apiary deployment produces.

An event-driven ingestion function consumes the queue, transforms each aggregate into a stored item, and writes it to a managed key-value store. The store uses a two-part key. The partition key is the sensor type, which groups all readings for a given quantity and makes the dominant query, "give me the recent windows for this signal," a single efficient lookup. The sort key concatenates the window-end timestamp with the apiary identifier. The timestamp gives a natural chronological order within each partition, and appending the apiary identifier is deliberate: when both apiaries close a window for the same signal in the same cycle, their two items share a partition key and a window-end time, and without the apiary identifier in the sort key the second write would overwrite the first. Concatenating the identifier keeps both apiaries' items distinct in one partition.

Beehive & Apiary Monitor | Fog-to-Cloud Deployment Topology

AWS Cloud (us-east-1)

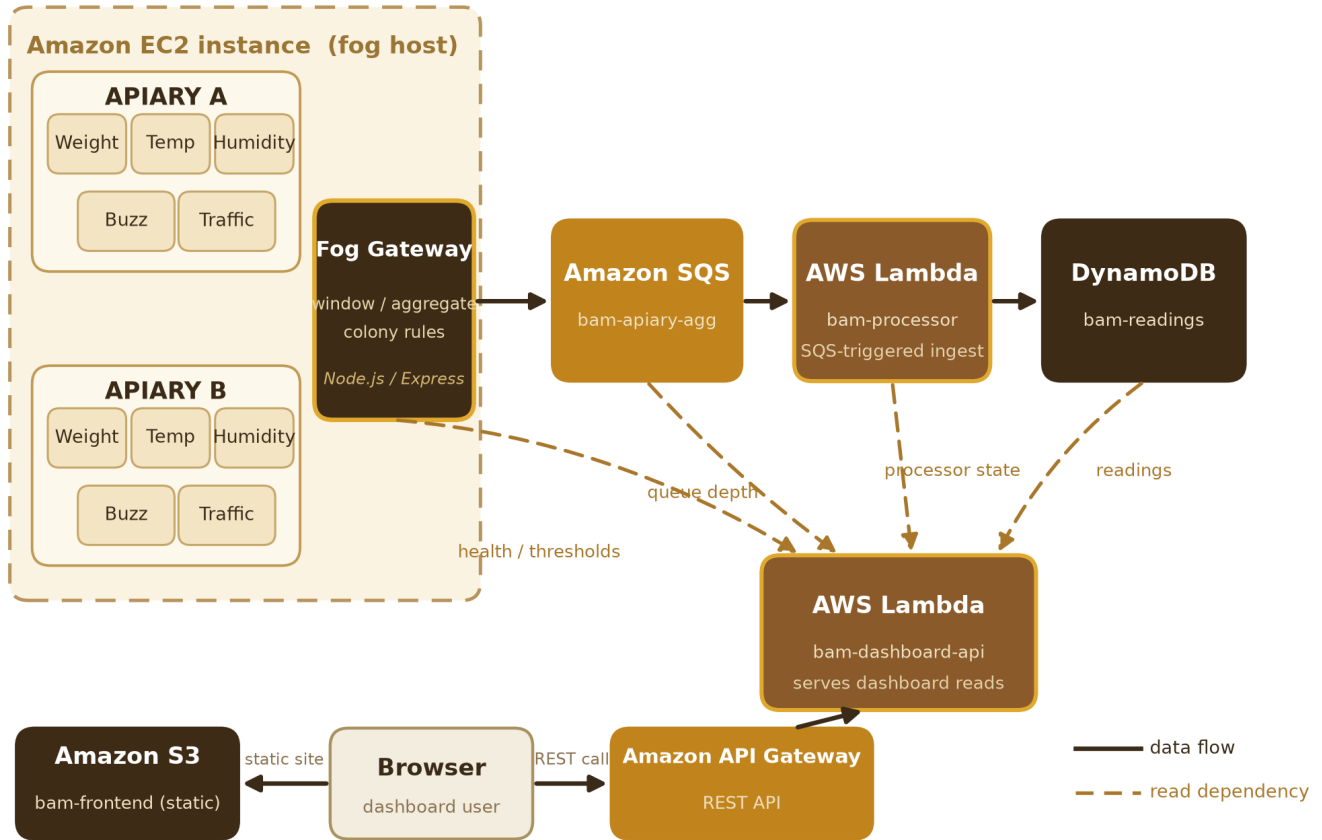


Fig. 1. Fog-to-cloud deployment topology. The fog gateway and the ten sensor processes run in containers on a single edge host at the apiary. Solid arrows are the ingestion data flow from the hives through the queue and the ingestion function into the store. Dashed arrows are the read dependencies the dashboard function assembles from the gateway, the queue, the ingestion function, and the store. The browser loads static assets from object storage and calls the dashboard programming interface through the managed gateway.

D. Read Tier: Interface, Colony-Health Narrative, and Frontend

The operator view is served by a second cloud function deployed independently of the ingestion function, so the read path and the write path scale and fail separately. This function answers a small read-only programming interface: an apiaries endpoint returning, for every apiary, the latest reading on each signal together with a synthesised colony-health summary; a per-apiary detail endpoint; a readings endpoint returning a recent series for one signal and apiary to draw the trend charts; a health endpoint; a back-end statistics endpoint; and a thresholds endpoint that proxies the gateway's published rules. The health endpoint reports four independent booleans, that the gateway is reachable, the queue is reachable, the ingestion function is active, and the pipeline is flowing, meaning the freshest stored window is recent, together with the age of the freshest window.

The distinctive element of this tier is the colony-health

narrative. For each apiary the function reads the recent weight and temperature windows and reduces them to two qualitative states: a weight trend, rising, falling, or steady, obtained by comparing the earliest and latest window means against a small threshold, and a brood thermal state, stable, fluctuating, or critical, obtained from the spread of recent temperature means and from whether any recent window already carries an alert. These states, together with the count of active alerts, are composed into a single plain-language sentence per apiary, so the dashboard can tell the beekeeper that, for instance, a colony's weight is rising while its brood temperature has breached a safe threshold. Turning telemetry into a judgement is exactly the kind of summarisation the edge-to-cloud split makes room for, and it is placed in the read tier because it depends on a short history of stored windows rather than on any single live reading.

The dashboard function is reached through a managed programming-interface gateway that exposes a public REST

endpoint. The static frontend, a single page, a stylesheet, the page script, and a locally vendored charting library, is served from object storage. The page renders a colony-health summary card per apiary carrying the narrative sentence and the live per-signal readings, an alert banner naming any firing condition, and a panel of window-average trend charts comparing the two apiaries; a header strip carries the four pipeline indicators. The page script re-fetches the apiaries view, the health endpoint, and the back-end statistics together on a short interval and repaints from each response. The frontend learns which programming-interface origin to call from a single inline value in the page that is substituted with the real gateway origin at upload time; in local development, where one process serves both the interface and the page, the value is left unset and calls default to the same origin.

E. Alternatives Considered

Each choice below was made for a stated reason.

A queue rather than a direct function call. The gateway could invoke the storage function directly and synchronously, removing the queue and one hop of latency. This was rejected: a synchronous call couples the edge to the availability and speed of the cloud function, so if the function is throttled, cold, or unavailable the gateway blocks or drops data and back pressure reaches the hives. The queue breaks that coupling, absorbs bursts, retries delivery on the consumer’s behalf, and lets the gateway consider an aggregate handed off the instant it is enqueued [8]. The extra latency is irrelevant on a window cadence; the durability and decoupling are what an intermittent apiary uplink needs.

A key-value store rather than a relational database. The access pattern is narrow and known in advance: append time-ordered aggregates, then read back the recent windows for a signal. A key-value store built for that pattern of high-volume writes and key-based reads gives predictable performance at scale without the operational weight of a relational engine, the trade-off the original Dynamo design articulated and the managed service inherits [9], [10]. The chosen key scheme turns the query the dashboard needs into a single partition lookup.

Event-driven functions rather than an always-on server. The ingestion and interface tiers run as event-driven functions, which suits a bursty and, for the interface, intermittent workload: the account pays per request rather than for idle capacity, and the platform scales the functions as load varies. The known cost is cold-start latency after a function has been idle [11]; here the effect is minor, because a steady stream of queue messages keeps the ingestion function warm and the dashboard’s short polling interval does the same for the interface.

F. Security Posture

The edge host exposes a single inbound port carrying only the gateway’s health and threshold endpoints; it has no remote-login key pair and is administered through the cloud provider’s systems-management channel, removing the most

TABLE II
AUTOMATED TEST COVERAGE BY MODULE

Module	Tests
Sensors	14
Fog gateway	64
Ingestion function	12
Dashboard interface	48
Total	138

common remote-access attack surface. Both cloud functions run under the account’s shared laboratory role, because the account cannot create new roles; the stated production intent is to split this into least-privilege roles, granting the ingestion function only the ability to receive and delete queue messages and write items, and the dashboard function only read-oriented permissions. The dashboard interface is intentionally unauthenticated because it exposes only aggregated, non-personal hive telemetry; because it is served from one origin and called from another, every response, including error responses, carries an explicit cross-origin header so a browser accepts the reply under the rules that govern cross-origin requests [12].

III. IMPLEMENTATION

The entire stack is implemented in Node.js with no build step and no TypeScript. The fog gateway is built on the Express web framework; the cloud clients for the queue, the store, and the functions come from the official provider software development kit for JavaScript. The dashboard’s trend charts are drawn with a charting library vendored into the project and served locally rather than from a content delivery network, so the page carries no third-party runtime dependency. For local development and continuous integration the full stack runs under Docker Compose against a local emulator of the cloud services, so the same code runs unchanged against the emulator or the real account by changing only endpoint and credential configuration.

A. Automated Test Suite

Testing uses the test runner built into the Node.js runtime, with no external framework. Cloud-facing code accepts an injected client, so the unit tests exercise it against small hand-written fake clients rather than a live service or even the emulator, keeping the suite fast, deterministic, and free of network dependence. A dedicated test also guards a subtlety of the serverless runtime: because that runtime invokes a handler with a completion callback as its third argument, the dashboard handler admits an injected-client argument only when it actually carries a store client, so the runtime’s callback is never mistaken for injected clients. The suite totals 138 tests across the four modules, distributed as shown in Table II. The fog module carries the largest share because it holds the windowing, the statistical reduction, and the colony rules.

B. Continuous Integration

A continuous-integration workflow runs on every push and every pull request as two dependent jobs. The first installs dependencies and runs the unit suites for all four Node.js modules. Only if that job passes does the second run: it brings up the emulator-backed Docker Compose stack, executes an end-to-end check that pushes data through the sensors, the gateway, the queue, the ingestion function, and the store, and then tears the stack down. Gating the integration job on the unit job keeps feedback fast in the common case, where a logic error is caught by the quick unit suite, while still exercising the assembled system before a change is accepted.

C. Infrastructure-as-Code Deployment

The system was deployed live to a real provider laboratory account in the North Virginia region. Provisioning is expressed as infrastructure as code and applied in an isolated workspace so that it plans only its own resources: a single apply created twenty-four resources with no manual command-line steps, the plan reporting twenty-four to add and none to change or destroy. The live resources comprise a key-value table, a message queue, the ingestion function and the dashboard function reached through a REST programming-interface gateway, an edge compute instance whose firewall admits only the single gateway port and which carries no login key pair, a fixed public address attached to that instance, and two object-storage buckets, one serving the dashboard frontend and one used for deployment staging. The application code and the infrastructure-as-code configuration are documented in the repository at [GitHub repository URL].

The system was then evaluated by measuring the running deployment, not by inspecting code alone; all figures below were read from the live endpoint.

IV. LIVE EVALUATION

A. End-to-End Liveness

Within about a minute of the edge instance finishing container start-up, the health endpoint reported all four indicators true, the gateway reachable, the queue reachable, the ingestion function active, and the pipeline flowing, and the freshest stored window was a few seconds old. A freshest-window age in single-digit seconds means a reading produced at a hive had already traversed the sensor, the gateway, the queue, the ingestion function, and the store and was being read back by the dashboard, all within seconds, the property that distinguishes a live pipeline from a set of separately deployed components.

B. Accumulation and Colony Narratives

The stored record count was polled repeatedly across the verification window and was observed to climb across successive polls, confirming that the ingestion function was consuming the queue and writing continuously. Over the same window the apiaries endpoint returned live per-apiary data for both apiaries, five current readings each with their window range and mean, and, crucially, a synthesised colony-health

sentence per apiary. During verification both apiaries' brood-nest temperature sat in the overheat band, and the pipeline behaved end to end as designed: the edge rule raised a brood-overheat alert on each window, the alerts propagated through the queue and the store to the read tier, each apiary's narrative reported that its brood temperature had breached a safe threshold while its weight was rising, and the dashboard's alert banner named both firing conditions. All static frontend assets were fetched successfully from object storage, the cross-origin header was present on the interface responses, and the inline interface origin in the page had been correctly substituted with the real gateway address at upload time.

Fig. 2 is a screenshot of the deployed dashboard rendered in a real browser. It shows the two colony-health summary cards with their narrative sentences and live readings, the alert banner naming the two brood-overheat conditions, the panel of window-average trend charts, and the header indicators reporting the pipeline healthy. The screenshot is evidence that the system renders correct, live, interpreted content to a human operator, not merely raw numbers.

C. Cost

The design keeps standing cost low. Every cloud service in the pipeline except the edge instance is billed per request and, at the volumes this deployment produces, stays within the provider's free tier: the queue, the two functions, the key-value store, the interface gateway, and the object storage all bill on use. The edge instance is the only continuously billed resource, and because the serverless dashboard and interface do not depend on it being up, only fresh readings and the gateway's own health reading do, it can be stopped while idle without taking the dashboard offline. The standing cost when idle is therefore essentially that of one small instance, and even that can be paused.

D. Limitations

Three limitations are stated plainly. First, the sensor and fog tier runs on a single edge instance, a single point of failure for that tier: if the instance stops, new readings stop, though previously stored data and the serverless read path remain available. A production apiary network would replicate the fog tier or place a gateway per site. Second, the laboratory account is session based, so only single-session behaviour was verified; multi-day stability and credential rotation over long runs were not exercised. Third, both cloud functions currently share one broadly scoped account role; splitting them into least-privilege roles is specified but was not possible in an account that cannot create new roles.

V. CONCLUSION

This work set out to move apiary monitoring from infrequent, disruptive inspection toward continuous, non-invasive, interpreted observation, and to do so with a distributed design that places computation where it belongs: windowing, statistical reduction, and colony-rule evaluation at a fog tier at the apiary, durable storage and elastic serving in the cloud, and a

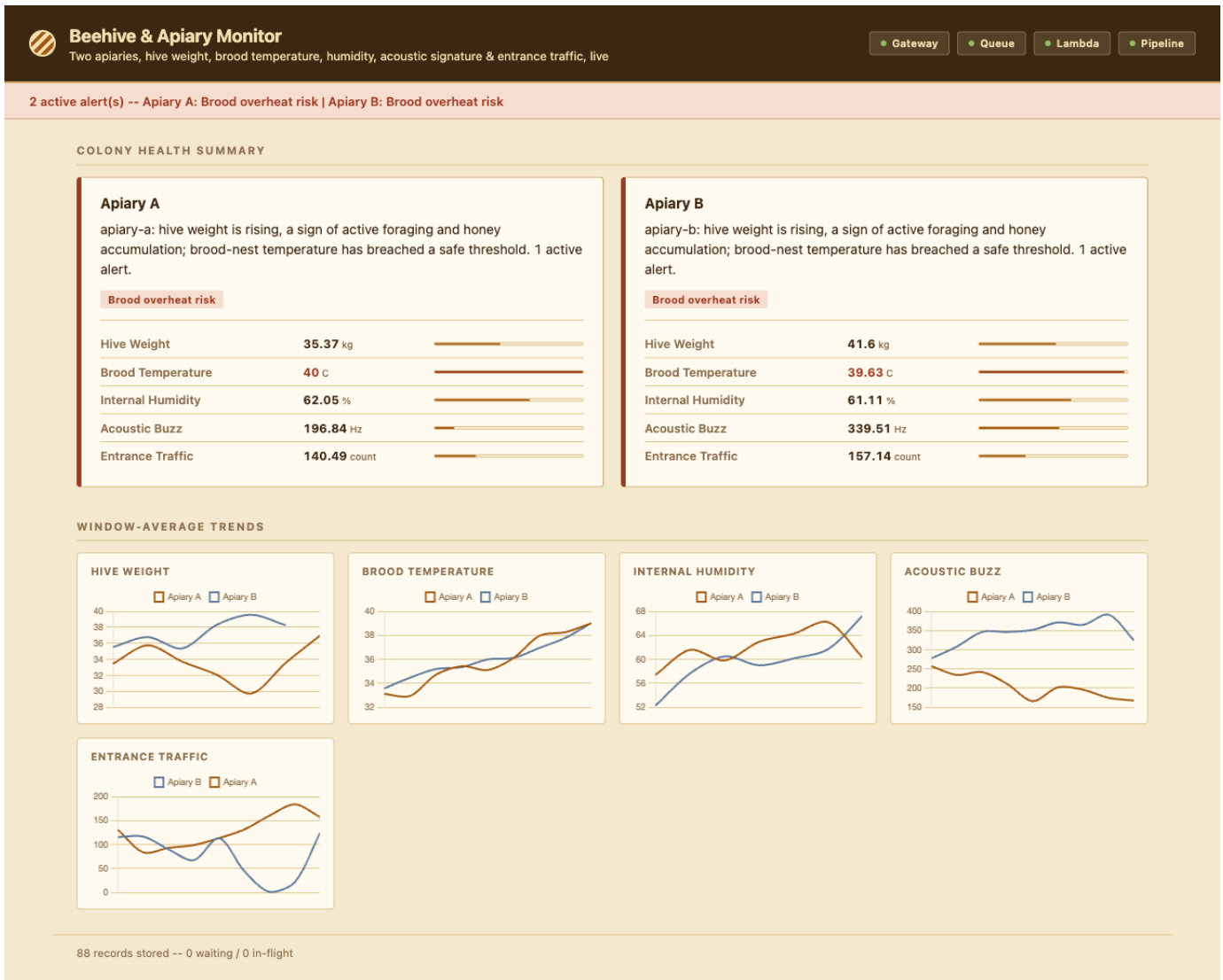


Fig. 2. The deployed operator dashboard rendered live in a browser. Each apiary card carries a plain-language colony-health sentence and its five live readings against their plausible ranges; the alert banner names the two firing brood-overheat conditions; the lower panel compares window-average trends across the two apiaries; the header carries the four pipeline health indicators.

synthesised health judgement at the read tier. The result is a working pipeline that watches two apiaries and five signals each, flags brood overheating, chilling, starvation, and the acoustic precursor of swarming at the edge, and presents both live readings and a plain-language colony-health narrative drawn from a real cloud deployment.

Several lessons stand out. The value of the message queue was not latency but resilience: decoupling an intermittent apiary uplink from the cloud consumer is what makes the pipeline tolerant of the conditions remote monitoring must survive. The sort-key design was a reminder that a small modelling decision, appending the apiary identifier to a timestamp, is the difference between two apiaries coexisting and one silently overwriting the other. The most rewarding lesson concerned where interpretation belongs: raw telemetry becomes useful to a beekeeper only when it is turned into a judgement, and the

natural home for that synthesis is the read tier, where a short history of stored windows is already at hand. Live verification confirmed the value of this end to end, when a real brood-overheat excursion travelled from an edge rule all the way to a named condition in the operator's narrative. Natural next steps are replicating the fog tier for high availability, splitting the shared role into least-privilege roles, and extending the edge rules from fixed thresholds toward trend-based detection, so that a slow decline in stores or a gradual shift in the acoustic signature is flagged from its trajectory rather than only when it crosses a line.

REFERENCES

- [1] A. Zacepins, V. Brusbardis, J. Meitalovs, and E. Stalidzans, "Challenges in the development of precision beekeeping," *Biosystems Engineering*, vol. 130, pp. 60–71, 2015.

- [2] W. G. Meikle and N. Holst, "Application of continuous monitoring of honeybee colonies," *Apidologie*, vol. 46, no. 1, pp. 10–22, 2015.
- [3] I. F. Akyildiz, W. Su, Y. Sankarasubramaniam, and E. Cayirci, "Wireless sensor networks: A survey," *Computer Networks*, vol. 38, no. 4, pp. 393–422, 2002.
- [4] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [5] F. Bonomi, R. Milito, J. Zhu, and S. Addepalli, "Fog computing and its role in the internet of things," in *Proc. 1st Edition of the MCC Workshop on Mobile Cloud Computing (MCC '12)*, 2012, pp. 13–16.
- [6] R. K. Naha, S. Garg, D. Georgakopoulos, P. P. Jayaraman, L. Gao, Y. Xiang, and R. Ranjan, "Fog computing: Survey of trends, architectures, requirements, and research directions," *IEEE Access*, vol. 6, pp. 47 980–48 009, 2018.
- [7] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep. Special Publication 800-145, 2011.
- [8] G. Hohpe and B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Addison-Wesley, 2003.
- [9] G. DeCandia, D. Hastorun, M. Jampani, G. Kakulapati, A. Lakshman, A. Pilchin, S. Sivasubramanian, P. Voshall, and W. Vogels, "Dynamo: Amazon's highly available key-value store," in *Proc. 21st ACM SIGOPS Symp. Operating Systems Principles (SOSP '07)*, 2007, pp. 205–220.
- [10] M. Elhemali, N. Gallagher, N. Gordon, J. Idziorek, R. Krog, C. Lazier, E. Mo, A. Mritunjai, S. Perianayagam, T. Rath, S. Sivasubramanian, J. C. Sorenson, S. Sothikul, D. Terry, and A. Vig, "Amazon DynamoDB: A scalable, predictably performant, and fully managed NoSQL database service," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '22)*, 2022, pp. 1037–1048.
- [11] L. Wang, M. Li, Y. Zhang, T. Ristenpart, and M. Swift, "Peeking behind the curtains of serverless platforms," in *Proc. USENIX Annual Technical Conf. (USENIX ATC '18)*, 2018, pp. 133–146.
- [12] A. Barth, "The web origin concept," Internet Engineering Task Force, Tech. Rep. RFC 6454, 2011.